

Implementation Plan: Consolidated Dashboard Refresh

Project: DevOps HQ Consolidated Report Feature
Prepared: 2026-04-09
Total Duration: 4-5 calendar days (2 developers in parallel)
Status: Ready for Sprint Planning

Overview

This plan breaks down the consolidated dashboard “Refresh” feature into frontend, backend, and integration work streams. Both teams work in parallel, with a 1-day integration buffer at the end.

Timeline Summary

Week 1:

Mon (Day 1-2)	Frontend + Backend Dev (parallel)
Wed (Day 3)	Integration begins
Thu (Day 4)	Testing + polish
Fri (Day 5)	Deploy to staging/prod

Start: This Friday (2026-04-11)
MVP Ready: Friday (2026-04-18)
Phase 1.5 (Redis): Following Monday (2026-04-21)

Work Breakdown Structure (WBS)

1. Frontend: UI Component & Integration

1.1 Component Skeleton (6 hours)

Owner: Frontend Lead

Duration: 1 day

☐

Create ConsolidatedReportDashboard component

☐

Set up React Query hooks

☐

Import mock API response (from backend doc)

☐

Build layout: summary cards, blockers section, task table

☐

Wire up Refresh button onClick handler

☐

Add loading/error states

Deliverable:

/src/components/DevOpsHQ/ConsolidatedReportDashboard.tsx

/src/hooks/useConsolidatedReport.ts (React Query hook)

/src/mocks/reportData.json (mock response)

Output: Component renders with mock data, button is interactive (no API yet).

1.2 Task Snapshot Table (4-5 hours)

Owner: Frontend Lead

Duration: 0.5-1 day

☐

Implement TanStack Table (React Table)

☐

Columns: ID, Title, Status, Owner, Risk, Updated, %Complete

☐

Sorting: by any column

☐

Filtering: by status, owner, risk level

☐

Expandable rows (subtasks/details)

☐

Cell styling (status badges, risk colors)

Deliverable:

/src/components/TaskSnapshotTable/TaskSnapshotTable.tsx
/src/components/TaskSnapshotTable/columns.tsx (column definitions)
/src/styles/table.css (table styling)

Output: Full-featured sortable/filterable table with mock data.

1.3 Blockers Display & Actions (3-4 hours)

Owner: Frontend Lead

Duration: 0.5 day

☐

Critical blockers card (pinned to top)

☐

Blocker details: title, reason, owner, risk, age

☐

Action buttons: [Unblock], [Escalate], [Reassign]

☐

Empty state: "No blockers" message

☐

Highlight blockers in task table (strikethrough or alert badge)

Deliverable:

/src/components/CriticalBlockers/BlockersCard.tsx
/src/components/CriticalBlockers/BlockerItem.tsx

Output: Blockers section displays and action buttons are wired (mock handlers).

1.4 Export Feature (4-6 hours)

Owner: Frontend Lead

Duration: 1 day

☐

CSV export: download report-2026-04-09.csv

- Columns: ID, Title, Status, Owner, Risk, Updated
- Rows: all visible tasks

☐

PDF export: styled report document

- Header: date, team, summary stats
- Body: blockers, task table
- Footer: page numbers, generated timestamp

☐

Slack export: generate preview, show “Share to Slack” button

- Message: summary cards + critical blockers
- Link to full report in HQ

Deliverable:

```
/src/utils/export/exportCSV.ts  
/src/utils/export/exportPDF.ts  
/src/utils/export/exportSlack.ts  
/src/components/ExportButtons.tsx
```

Output: All three export formats functional with mock data.

1.5 Connect to Real API (2-3 hours)

Owner: Frontend Lead

Duration: 0.5 day (starts Day 3, after Ward’s API is ready)

☐

Update useConsolidatedReport.ts hook to call real API

☐

Remove mock data, use response from POST /api/v1/dashboard/consolidated-report

☐

Wire up filters (team, status, owner) to API request

☐

Implement React Query caching (TTL: 3-5 min)

☐

Add error boundary + error toast notifications

☐

Implement retry logic (exponential backoff)

Deliverable:

```
/src/api/consolidatedReportAPI.ts (API client)  
/src/hooks/useConsolidatedReport.ts (updated, now hits real API)
```

Output: Dashboard pulls live data from backend.

1.6 Testing & Polish (2-3 hours)

Owner: Frontend + QA

Duration: 1 day (Day 4)

☐

Unit tests: TanStack Table sorting/filtering

☐

Integration tests: mock API, test render flow

☐

E2E tests: click Refresh → see data → export CSV

☐

Responsive design: mobile, tablet, desktop

☐

Accessibility: keyboard navigation, ARIA labels

☐

Performance: Lighthouse score >80, bundle size OK

Deliverable:

/src/__tests__/ConsolidatedReportDashboard.test.tsx
/e2e/consolidated-report.spec.ts (Playwright or Cypress)

Output: All tests passing, no TypeScript errors, ready for production.

2. Backend: API Endpoint & Data Aggregation

2.1 Database Schema Setup (3-4 hours)

Owner: Ward (Backend)

Duration: 0.5 day (Day 1)

☐

Review existing tasks table schema

☐

Create blockers table (if not exists)

- Fields: id, task_id, reason, owner_id, risk_level, created_at, resolved_at

☐

Create task_owners_view (for easier queries)

☐

Add indexes on: task_id, status, assigned_to, risk_level

☐

Set up RLS policies (Supabase) for team/user isolation

☐

Write migration scripts

Deliverable:

/migrations/001_blockers_table.sql
/migrations/002_rls_policies.sql
/supabase/schema.sql (updated)

Output: Tables exist, RLS configured, indexes created.

2.2 Data Aggregation Logic (4-6 hours)

Owner: Ward (Backend)

Duration: 1 day (Day 1-2)

- ☐ Write query to get task counts by status (using aggregation)
- ☐ Write query to get all blockers (with owner info, age calculation)
- ☐ Write query to rank blockers by risk + age
- ☐ Calculate risk level (auto-score based on age/priority or pull from field?)
- ☐ Calculate velocity (completed tasks per week)
- ☐ Write query to get task snapshot (top 50, with owner details)
- ☐ Test queries with large datasets (100+ tasks)

Deliverable:

/src/services/reportService.ts (aggregation logic)
/src/db/queries/consolidatedReport.sql (raw SQL or ORM)
/tests/reportService.test.ts (unit tests)

Output: Aggregation functions tested, return correct data structure.

2.3 API Endpoint Implementation (3-4 hours)

Owner: Ward (Backend)

Duration: 0.5 day (Day 2)

- ☐ Create Express/Hono route handler
- ☐ Implement request validation (Zod schema)
 - filters: { team?, status?, owner?, riskLevel?, limit?, offset? }
- ☐ Call aggregation service
- ☐ Format response per API contract (see ARCHITECTURE_RECOMMENDATION.md)
- ☐ Add JWT auth check (verify user has access)

☐

Add request logging

☐

Return proper error responses (400, 401, 500)

Deliverable:

/src/routes/dashboard.ts (or dashboard.controller.ts)
/src/schemas/consolidatedReportRequest.ts (Zod validation)
/src/middleware/authCheck.ts (if not exists)

Output: Endpoint callable via curl/Postman, returns correct JSON structure.

2.4 Caching Layer (2-3 hours)

Owner: Ward (Backend)

Duration: 0.5 day (Day 2)

☐

Set up Redis client connection

☐

Implement cache key generation: `report:${userId}:${hash(filters)}`

☐

Check cache before DB query (cache-aside pattern)

☐

Store result in Redis with 5-min TTL

☐

Handle cache misses + store fresh data

☐

Implement cache invalidation (on task/blocker updates)

Deliverable:

/src/services/cacheService.ts (cache helpers)
/src/routes/dashboard.ts (updated with caching logic)

Output: Cache working, hit rate measurable, latency <1s for cached requests.

2.5 Testing & Documentation (2-3 hours)

Owner: Ward + QA

Duration: 1 day (Day 3-4)

☐

Unit tests: aggregation functions

- ☐ Integration tests: API endpoint with real DB (test fixtures)
- ☐ Load test: 100 concurrent requests, measure P95 latency
- ☐ Error cases: missing data, timeout, invalid filters
- ☐ API documentation: Swagger/OpenAPI spec
- ☐ Performance benchmarks: record baseline latency

Deliverable:

/tests/routes/dashboard.test.ts
/docs/API.md (or Swagger file)
/benchmarks/report-latency.json (baseline metrics)

Output: All tests passing, load test shows <5s latency at 100 req/s, API documented.

3. Integration & Deployment

3.1 End-to-End Testing (2-3 hours)

Owner: Frontend + Backend + QA

Duration: 0.5 day (Day 4)

- ☐ Frontend calls real backend API
- ☐ Data flows correctly: API response → component render
- ☐ Filters work: click “Status: Blocked” → table filters
- ☐ Exports work: CSV, PDF, Slack all functional
- ☐ Cache works: second refresh is instant
- ☐ Error handling: API error → shows toast notification

Deliverable:

/e2e/consolidated-report-full.spec.ts (end-to-end tests)

Output: Smoke test passes, feature is integrated.

3.2 Staging Deployment (1-2 hours)

Owner: DevOps / Ward

Duration: 0.5 day (Day 4)

☐

Deploy backend to staging environment

☐

Deploy frontend to staging

☐

Verify both environments talking to correct databases

☐

Manual testing in staging: click, filter, export

☐

Verify logs + monitoring are working

Output: Feature live on staging, ready for user acceptance testing.

3.3 Production Release (1 hour)

Owner: DevOps / Ward

Duration: 0.25 day (Day 5)

☐

Feature flag enabled (if using feature flags)

☐

Deploy to production

☐

Monitor error rates + latency for 1 hour

☐

Verify real data flowing (spot-check sample tasks)

☐

Document deployment + rollback procedure

Output: Feature live in production.

Detailed Timeline

Day 1 (Friday, 2026-04-11): Kickoff & Foundation

Parallel work by Frontend + Backend

Frontend (6-8 hours): - [] Component skeleton + mock data - []
React Query hook setup - [] Layout: summary cards, blockers,
task table - [] Refresh button wired (to mock API)

Backend (7-8 hours): - [] Database schema: blockers table,
indexes, RLS - [] Data aggregation: write + test queries - [] Start
API endpoint implementation

Sync meeting: End of day, review progress, identify blockers.

Day 2 (Monday, 2026-04-14): Core Features

Parallel work continues

Frontend (6-7 hours): - [] Task snapshot table: sorting +
filtering complete - [] Blockers display: action buttons wired - []
Export feature: CSV + PDF + Slack

Backend (6-7 hours): - [] API endpoint complete + tested with
curl - [] Caching layer implemented (Redis) - [] Error handling +
validation

Sync meeting: Midday, Ward demos API, Frontend confirms
contract matches.

Day 3 (Tuesday, 2026-04-15): Integration Begins

Frontend now integrates with real API

Frontend (4-5 hours): - [] Connect component to real API - []
Implement React Query caching - [] Error boundaries + toast
notifications - [] Test with real data from backend

Backend (2-3 hours): - [] Finalize caching + performance
tuning - [] Load testing (capture baseline metrics) - [] API
documentation (Swagger)

Sync meeting: End of day, integration checkpoint. Any data
mismatches?

Day 4 (Wednesday, 2026-04-16): Testing & Polish

Both teams focus on quality & deployment readiness

Frontend (3-4 hours): - [] Unit + integration tests - []
Responsive design + accessibility - [] Staging deployment - []
Manual testing in staging

Backend (2-3 hours): - [] E2E tests (API + DB) - [] Performance benchmarks - [] Staging deployment - [] Monitor + verify

Sync meeting: Pre-prod checklist review. Go/No-go for production?

Day 5 (Thursday, 2026-04-17): Production Release

Calm, methodical final day

Deployment (1-2 hours): - [] Deploy to production - [] Monitor errors + latency for 1 hour - [] Verify real data - [] Celebrate! ☑

Post-release (1-2 hours): - [] Document feature (user guide, changelog) - [] Gather feedback from team - [] Log any follow-ups for Phase 1.5

Resource Allocation

Role	Person	Allocation	Start	End
Frontend Dev	[TBD]	100%	Fri 4/11	Thu 4/17
Backend Dev	Ward	100%	Fri 4/11	Thu 4/17
QA / Testing	[TBD]	50%	Wed 4/15	Thu 4/17
DevOps	[TBD]	25%	Tue 4/14	Thu 4/17
Product Lead	[TBD]	10%	Fri 4/11	Thu 4/17

Risk & Mitigation

Risk 1: API Latency > 5s

Probability: Medium

Impact: Users wait too long for refresh

Mitigation: - Daily performance benchmarks (starting Day 2) - If >5s, optimize indexes + add Redis caching - Fallback: paginate task results (only return 50 tasks)

Risk 2: Data Schema Mismatch

Probability: Low

Impact: API response doesn't match frontend expectation

Mitigation: - Share API contract

(ARCHITECTURE_RECOMMENDATION.md) on Day 1 - Sync meeting mid-Day 2 to confirm - Use mock API response in frontend until real API ready

Risk 3: RLS Policy Breakage

Probability: Low

Impact: Users see data they shouldn't

Mitigation: - Ward reviews RLS policies carefully - Test with multiple user roles (admin, regular, guest) - Staging environment has realistic test data

Risk 4: Integration Delay

Probability: Medium

Impact: Frontend/backend dev finishes early, idle time

Mitigation: - Frontend starts with mock data (not blocked by backend) - Backend exposes mock API endpoint on Day 1 - Both teams can work in parallel with clear contract

Risk 5: Scope Creep

Probability: High

Impact: Phase 2 features leak into MVP

Mitigation: - Lock scope: MVP = Refresh button + report display + CSV export only - Phase 2 = WebSocket, Slack, PDF, auto-refresh (separate tickets) - Use feature flags to separate phases

Acceptance Criteria

MVP Complete ☐

☐

Refresh button on DevOps HQ header

☐

Click button → API call → data updates in <5s

☐

Dashboard shows: summary cards, blockers, task table

☐

Sorting + filtering works

☐

Export to CSV works

☐

Error handling (API down, timeout, etc.)

☐

- ☐ All tests passing
- ☐ Live in production on 2026-04-17

Success Metrics

- ☐ Task status breakdown visible
 - ☐ Blockers surfaced and highlighted
 - ☐ Owner assignment visible on all tasks
 - ☐ Risk level color-coded (red/yellow/green)
 - ☐ Cache working (80%+ hit rate on second refresh)
 - ☐ Zero security issues (RLS enforced, auth validated)
 - ☐ Zero data loss (all tasks + blockers visible)
-

Handoff Points for Ward (Backend)

Deliverables Ward Owns:

1. ☐ **Database Schema** (blockers table, indexes, RLS policies)
2. ☐ **API Endpoint** (POST /api/v1/dashboard/consolidated-report)
3. ☐ **Data Aggregation Service** (queries + business logic)
4. ☐ **Caching Layer** (Redis integration)
5. ☐ **API Documentation** (Swagger/OpenAPI)
6. ☐ **Error Handling** (400, 401, 500 responses)
7. ☐ **Testing** (unit + integration tests)
8. ☐ **Load Testing** (baseline metrics P95 latency)
9. ☐ **Staging Deployment** (verify in non-prod first)
10. ☐ **Production Deployment** (including rollback procedure)

Questions for Ward (Answered by End of Day 1):

1. How is risk level calculated? (Auto from age/priority, or manual field?)
2. How is “blocked” status set? (Manual flag, or auto-detect from deps?)
3. Is owner single or multiple? (If multiple, how to surface in report?)
4. Should report scope to one team, or org-wide?

5. What's the current max task count? (Helps estimate query performance)
 6. Can we use Supabase Realtime, or stick to polling?
 7. Is there existing caching infrastructure (Redis), or do we set up new?
-

Success Definition

MVP is successful when: 1. Feature deploys to production with zero critical bugs 2. Task status, blockers, owner, and risk visible in consolidated report 3. Refresh button generates fresh data in <5s 4. Cache reduces second refresh to <1s 5. All three export formats work 6. Team reports it's useful + starts using it daily

Appendix: Environment Setup

Frontend Prerequisites

```
npm install react-query tanstack-table axios csv-download jspdf
npm install -D @testing-library/react vitest
```

Backend Prerequisites

```
npm install express zod supabase redis dotenv
npm install -D jest ts-node typescript
```

Local Development

```
# Backend: Start local Redis + Supabase
```

```
docker-compose up -d
```

```
# Backend: Run dev server
```

```
npm run dev
```

```
# Frontend: Start dev server
```

```
npm run dev
```

```
# Frontend: Call local API
```

```
# API endpoint: http://localhost:3001/api/v1/dashboard/consolidated-report
```

Status: Ready for sprint planning

Next Step: Ward reviews & approves architecture, schedules kickoff meeting